

typetrainer

Development log

How the practical part of the engineering thesis was built together with an AI coding agent, from the first idea to a working, published application. 3–13 October 2026.

Reference material for writing the thesis · Lublin University of Technology · @69inoi69
Live: 69inoi69.github.io/typing-trainer · Code: github.com/69inoi69/typing-trainer

How to use this document. These are working notes, not thesis text. They record what was decided, why, and in which order, so the thesis can describe the process accurately. Everything here comes from the conversation history and the Git history of the repository. The student's own messages were written in Russian; where they are quoted, they are paraphrased in English. The theory chapters still have to be written in your own words (anti-plagiarism check), and the formulas should be taken from *docs/ALGORITHM.md*, which always describes what the code really does.

OVERVIEW

At a glance

What	A web application for learning touch typing. It looks and feels like Monkeytype, but it teaches: it tracks every letter's mistakes and speed and builds each exercise from real words around the user's weakest letters.
Stack	Go (standard library only) · plain HTML, CSS and JavaScript (no frameworks, no npm, no build step) · in-memory storage behind a *Store* interface, PostgreSQL planned · WebAssembly build for GitHub Pages.
Languages	English, Polish, Ukrainian, Russian; 2000 real words each; own keyboard layout and own progress per language.
Size	About 1,900 lines of Go, 2,700 lines of frontend, 32 automated tests, 4 documentation files (about 660 lines).
Timeline	Idea and first prototype: 3 Oct (evening). Tools, Git and GitHub: 3 Oct (night), first commit 4 Oct 01:21. Main development day: 12 Oct, 11:43–22:39 (31 commits). This log: 13 Oct.
Roles	Student: idea, product decisions, visual judgement, testing on a real PC and phone, final say. AI agent (Claude): implementation, tests, documentation, browser checks, deployment, plain-language explanations.

Timeline

When	Phase	Result
3 Oct, ~16:30	Idea	Feasibility discussion; the student's prompt with stack, features and architecture rules.
3 Oct, ~16:40	Prototype 1	Working typing screen, keyboard with finger zones, simple adaptive weights, tests, README.
3 Oct, ~16:45–17:05	The algorithm	Student's clarification (weights, temperature, one mode, real words); version 2 with EMA, softmax, gradual unlocking; docs/ folder.
3 Oct, night	Workflow	VS Code profile, official Claude Code extension, Git for Windows, GitHub repository, AGENTS.md rules.
12 Oct, 11:43–12:33	Foundations	Smooth caret and stats screen; WebAssembly site on GitHub Pages; four languages; interface in four languages.
12 Oct, 13:05–16:29	Feel and UI	Rewards and streak, Tab without restart, start screen, icon toolbar, hands, hands-on intro, reset, status pills.
12 Oct, 17:09–18:52	Phones	Phone layout, keyboard hidden by default, contrast, no tap highlight.
12 Oct, 19:02	Data	Export / import of all statistics as a JSON file; first look at real data.
12 Oct, 19:23–22:28	Consistency	One look for all controls, favicon, logo, raw wpm, heatmap on the stats screen, keyboard presets that always fit, inline messages, new hands.
12 Oct, 22:39	Presentation	README for visitors with screenshots; full user guide in docs/.

3 OCTOBER

1. Starting point: the idea and the first prototype

The final semester leaves most of the time for the thesis. The student had already chosen the stack he wanted to learn (Go for the logic, plain JavaScript for the page, PostgreSQL for data) and wanted to test, on a real task, how well an AI agent could build it, before deciding how to work.

The idea, in the student's words (paraphrased): it should look and be as pleasant to use as Monkeytype, but it must *teach* touch typing. It should show where to put the fingers. It needs its own algorithm that picks words. Statistics, a database, pleasant animations. One language (English) is enough for the thesis.

The first prompt. The student brought a prepared specification (written with the help of another chat). Its main points are worth keeping, because the project still follows them:

- Backend in Go with the standard library only; frontend in plain HTML, CSS and JavaScript without a build step.
- Storage in memory, but behind an interface, so PostgreSQL can be added later without rewriting anything.
- An on-screen keyboard coloured by finger zones that highlights the next key.
- A server-side adaptive engine: per-key attempts, errors and time; a difficulty score; weighted random selection.
- Architecture rules: engine knows nothing about HTTP or the database, layouts are data, routes start with /api/v1/, all tunable numbers in one place, unit tests for edge cases (a key with no statistics yet).
- Explicitly not now: user accounts, PostgreSQL, extra features. "The smallest thing that works."

Prototype 1 came back within minutes: the typing screen, the coloured keyboard, a placeholder formula (difficulty from error rate plus normalised reaction time) and 9 engine tests. A browser version of the same logic could be tried directly on the phone. The student's reaction was that it already looked and worked like Monkeytype.

3 OCTOBER

2. Defining the algorithm

After trying prototype 1, the student explained what the algorithm must really do (paraphrased):

> *“Words should be picked dynamically depending on where the person makes mistakes, how often, which letters are bad. There should be weights, something like a temperature. It should analyse the typing and give exactly the words where you struggle, so you train your weak spots right away. One mode: switching difficulty by hand is not nice, it should decide by itself. And maybe a short, not boring intro for people who cannot type at all.”*

He also asked whether this needs “its own AI”. The answer that shaped the thesis: **no neural network is needed**. Plain statistics can do it (keybr.com works this way too), and for a thesis this is an advantage: every decision can be written as a formula and defended, instead of “the model decided”. The weaknesses of prototype 1 were named and fixed in version 2:

Problem in prototype 1	Change in version 2
Lifetime error rate: old mistakes are never forgotten.	Exponential moving average (EMA) per letter for errors and time; recent presses count more.
A word was chosen through one random letter, the effect was weak.	First a focus letter, then a word that contains it and preferably other weak letters.
Linear weights.	Softmax with a temperature T: P ~ exp(difficulty / T). One parameter controls how hard it pushes on weak letters.
Beginners see all 26 letters at once.	Gradual unlocking by letter frequency; a new letter opens when all open ones are mastered. Experienced typists open everything.
One correct press makes a letter look perfect.	Shrinkage towards a prior (assume a new letter is fairly hard until there is evidence).

Two further product decisions came from the student here:

- **Real words only.** He disliked keybr's letter combinations; he had learned to type himself by retyping Wikipedia articles. Real words are easier to read and keep motivation up.
- **Documentation for the thesis from day one:** how the algorithm works and why, plus a reading list. A market analysis (Monkeytype, keybr and others) was deliberately postponed until he has tried those sites himself.
- Accounts, profile pages and leaderboards were mentioned as a possible future and explicitly kept out of scope.

The result of the evening was version 2 with 17 engine tests and the *docs/* folder: *ALGORITHM.md*, *ARCHITECTURE.md*, *LITERATURE.md*. Today's parameter values (all in *internal/config/config.go*):

```
per letter (EMA, a = max(0.1, 1/n)):  ErrEMA and TimeEMA (time from clean presses only)
raw         = 0.6 * clamp(ErrEMA / 0.25) + 0.4 * clamp((TimeEMA - 400 ms) / 600 ms)
difficulty  = (n * raw + 5 * 0.7) / (n + 5)          prior 0.7, worth 5 samples
P(letter)   ~ exp(difficulty / 0.25)                 the focus letter
P(word)     ~ exp(mean difficulty of its letters / 0.25) * 0.3^(uses in this exercise)
mastered    = n >= 20 and ErrEMA <= 5 % and TimeEMA <= 400 ms (30 wpm)
unlocking   : all open letters mastered -> the next letter (by frequency) opens
first lesson: at least 6 letters and at least 30 real words
```

3 OCTOBER, NIGHT

3. Setting up the way of working

The same night the project moved from a downloaded zip to a proper setup on the student's laptop (Windows, PowerShell). The advice was that a good AI workflow today means the agent works directly in the project folder, and that the tool matters less than a few habits around it:

- **Git and GitHub from the first day**, so every working step is a save point and any mistake can be undone.
- **VS Code with a clean profile** for the thesis (only Go, the official Claude Code extension, Mistral Vibe, SQLTools with the PostgreSQL driver) instead of switching to VSCodium.
- **One agent per task**, never two editing the same files at once.
- **A rules file in the repository** that every agent reads at the start of a session.

Small practical problems were solved on the way and are typical for a first setup: port 8080 was taken by another program (the server moved to port 3000, which is exactly why all settings live in one config file); VS Code did not see Git until it was restarted; the GitHub repository had to be created before the first push. The first commit, *Add agent rules*, landed on 4 October at 01:21.

The project rules (AGENTS.md)

The rules file turned the student's requirements into instructions every agent follows. Its main points:

- Who the agent works with: explain every change in simple words; talk in Russian, write code and docs in English; work in small steps and show a plan for anything bigger than one feature; never commit or push without being asked.
- Stack that must not change without asking: Go standard library, plain JavaScript, no frameworks, no npm.
- Architecture: the engine is pure logic; all data goes through *Store*; every tunable number in *config.go*; layouts and word lists are data; keypress handling stays light (one layout read per frame, CSS or Web Animations only).
- **The math in model.go is the core of the thesis: formulas and parameter values must not change without asking.** When behaviour changes, *ALGORITHM.md* and the tests change in the same task.
- Not now: accounts, leaderboards, profile pages, themes, more languages.
- Definition of done: gofmt, go vet and go test pass, edge cases tested, docs updated, a short summary for the student.

The rules grew with the project (four languages, i18n, visual rewards in *fx.js*, the two builds), but the core stayed the same. Later the student asked for changes to be pushed straight to *main*; he then runs *git pull* locally.

12 OCTOBER

4. The main development day

On 12 October the project went from a local prototype to a published, four-language application in 31 commits. The student worked partly at home and partly from his phone during classes, testing the live site and sending screenshots with comments. The phases below follow the Git history.

4.1 Polishing the core (11:43)

- The caret became one element that glides between letters; key presses animate with the Web Animations API, so typing stays fast.
- Statistics moved to their own screen with an animated type / stats switch: totals, a speed chart, every letter with its difficulty.
- A common beginner trap was explained and handled: opening *index.html* by double-click (file://) cannot work, the page must come from the server. The page now says so.

4.2 An online version without a server (11:55, 12:25)

GitHub Pages can only host static files. Instead of rewriting the logic in JavaScript, the **same Go code is compiled to WebAssembly** and runs in the browser: *main.go* starts the normal server, *main_wasm.go* exposes the API to JavaScript, a small shim sends */api/v1/* requests to it, and shared setup lives in *setup.go*. Every push to *main* runs the tests and republishes the site. A caching bug (browsers mixing old and new files after a deploy) was fixed by versioning every asset with the commit id.

4.3 Four languages (12:21, 12:33)

- Word lists: the 2000 most frequent words per language from FrequencyWords (OpenSubtitles 2018, CC BY-SA 4.0), filtered by a documented script (*tools/wordlists/build.py*) instead of hand-made lists, so the thesis can describe how they were made.
- Layouts as data: Polish Programmers (AltGr letters), Ukrainian and Russian ЙЦУКЕН. Languages are configured in *data/languages.json*; a new language needs no code.
- **Each language keeps its own progress**: being good at English says nothing about Ukrainian.
- The engine works on letters (runes), not bytes, for ą and Cyrillic; formulas unchanged.
- Caps Lock and a wrong system layout are detected and **not counted as mistakes**; the user gets a hint instead.
- The whole interface follows the practice language. Finger hints use the grammatical case each language needs (for example “lewym małym palcem”, “лівим мізинцем”).

4.4 Making it feel alive (13:05–13:51)

- Visual rewards in a separate file (*fx.js*): confetti, counting numbers, small bumps, a clean-word streak. Rule: rewards never change what is measured or sent.
- **Tab does not restart the exercise.** A deliberate product decision: restarting throws away exactly the mistakes the trainer learns from. Tab answers with a kind message, and the number of resisted restarts is shown at the end.
- A start screen: the logo types itself, a short phrase appears, the logo flies into the header. It also covers the WebAssembly loading.
- A centred layout with an icon toolbar and tooltips; two hands above the keyboard that light up the finger for the next key.
- A hands-on first-visit intro instead of a text to read: press the eight home-row keys (detected by physical key position, so it works with any system layout), press space, start. Experienced typists can open all letters.
- Shortcuts: Shift + Tab for type / stats, Alt + 1…4 for the language. The header switches stay exactly centred in every language.

4.5 Details that came from using it (14:55–16:29)

From here most changes were the student's reactions after using the app. Examples of how feedback became decisions:

Student's feedback (paraphrased)	What changed
Notifications overlap at the bottom; they should be top right, stay longer, with a progress bar, and look different per type (but not random).	A stack of messages with a time-left bar; six kinds (record, new letter, mastered, streak, Tab resisted, Tab) with their own colour, icon and small animation.
“A reset button would be nice, to get back to the starting state quickly while there is no database.”	New Store method and API route; a two-click bin button that resets all languages and browser settings.
“When I press Tab, the caret flies across the screen.”	Bug found: two animations multiplied the same CSS transform. The caret position moved to a separate property.
“Make everything smooth, without fanaticism; the layout warning disappears abruptly.”	Folding, fading and growing transitions for warnings, keyboard, intro and language switch.

Student's feedback (paraphrased)	What changed
“I like the whole interface to stay in place; the warning pushes everything up.” Status text should be icons.	Status pills under the letters; warnings appear on top of them in one fixed slot, so nothing moves.

4.6 Phones (17:09–18:52)

- Tested by the student on his phone during class: a big empty gap under the header, notifications covering the header.
- Fixes: phone-only layout rules, compact messages, the on-screen keyboard **hidden by default on touch devices** (there are no physical keys to look at), content centred when it is hidden.
- Higher-contrast switches and no blue flash when tapping.

4.7 Getting data out (19:02)

The student asked whether he should send his statistics later for debugging. Because progress lives only in memory (and only in the open tab on the website), an **export / import** of all statistics as one JSON file was added: full history, per language, validated on import (impossible numbers or foreign letters are rejected and nothing changes). It keeps progress across restarts, lets the data be analysed, and is the natural first format for moving to PostgreSQL. The first exported file (two exercises typed on a phone) was checked: all counters added up, and it showed why conclusions need about 20 presses per letter.

4.8 One consistent look (19:23–22:28)

The evening was spent on consistency and clarity. Several features went through more than one version until they felt right:

Feature	Iterations
Keyboard size	Slider 0–140 % → buttons − / 100 % / + → presets off · S · M · L that are shares of the largest keyboard that fits the window, so the typing screen never scrolls.
Hands	Bars per finger → SVG hand with palm and outlines → one soft shape with fully coloured fingers → thumbs pointing up and fingers inside the palm, “so it looks natural”.
Notifications	Queue at the bottom → stack top right with a time bar → part of the interface: a message temporarily replaces the status line under the letters.
Heatmap	Red overlay on the typing keyboard → the student did not understand what he was looking at → a grey keyboard on the stats screen where red means difficulty, with a legend and numbers on hover.
Streak	Small counter → same size and line as wpm / acc, with a flame that gets “hotter” at 10, 25 and 50 clean words.

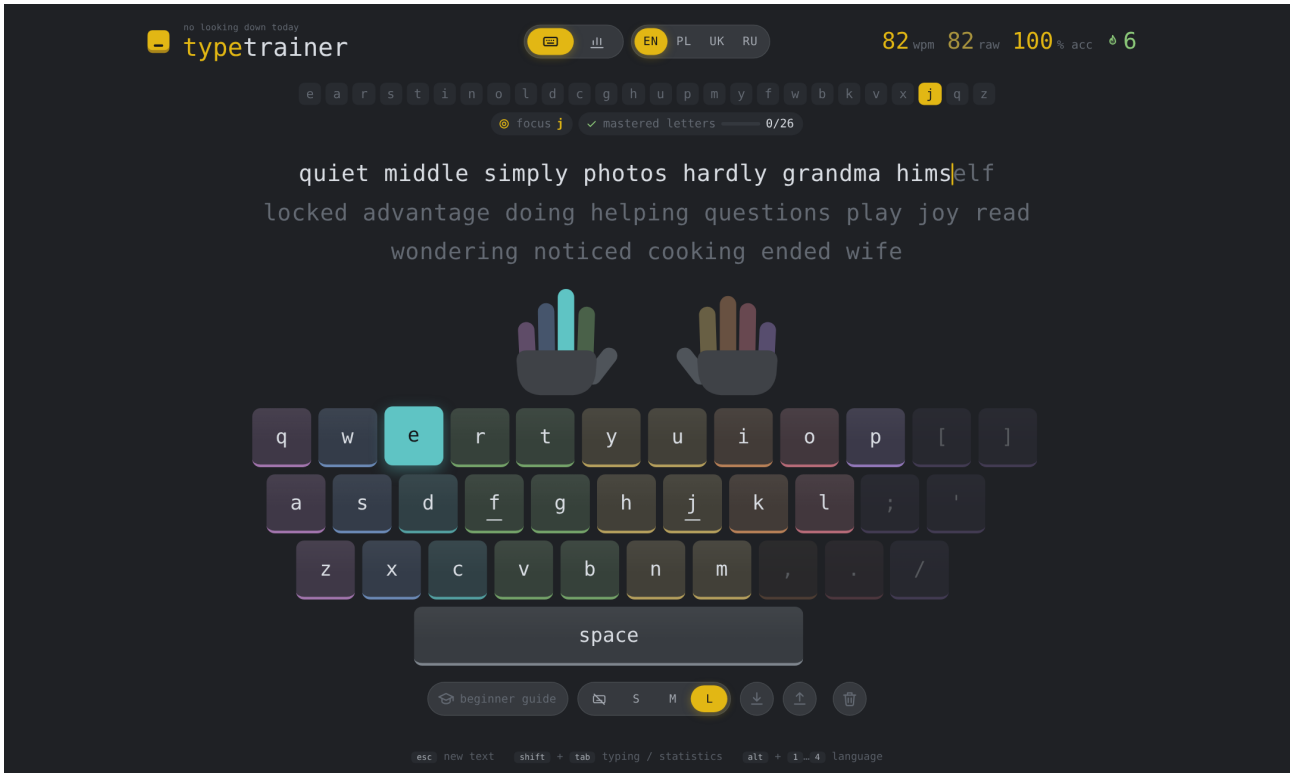
- Other additions: a tab icon (a yellow key with the home-row bump), a logo with a small random motto that the start-screen phrase flies onto, **raw wpm** next to wpm (all keypresses, shown only, never stored), keyboard shortcut hints at the bottom, a “beginner guide” button, a flat background.
- A question the student asked about the algorithm (“what algorithm and which words did you take?”) was answered with a worked example on his own data, which is a good format for the thesis too.

4.9 Presenting the project (22:39)

The README was rewritten for a random visitor of the public repository: logo, one-line pitch, a link to the live site, screenshots, why it exists, what it does, the algorithm in 30 seconds, quick start, credits. The detailed description of every screen moved to *docs/USER-GUIDE.md*.

Side work: the thesis schedule (harmonogram)

In the same period the official thesis schedule (PDF for the supervisor) was rewritten twice. The student wanted an iterative, flexible plan instead of fixed dates for each task, with the writing of the theoretical and practical parts as their own stages, no milestones, and signature fields for the student and the supervisor.



The typing screen at the end of 12 October: unlocked letters with the focus letter, status line, text, hands and keyboard, the off · S · M · L switch and shortcut hints.

FOR THE DESIGN CHAPTER

5. Key decisions and why

Decision	Alternative	Why (and where it is documented)
Real words only	Generated letter combinations (keybr)	Closer to real typing, easier to read, more motivating. Student's decision. ALGORITHM.md §2.
One mode, no difficulty switch	Manual levels (200 / 1K / 5K words)	The algorithm decides; nothing to configure. Student's decision. AGENTS.md.
Statistics, not a neural network	A trained model	No training data needed, every step explainable and defensible, instant. ALGORITHM.md §2.
EMA per letter	Lifetime averages	Skill changes; old mistakes should fade (half-life ≈ 6.6 occurrences). §4.
Prior / shrinkage	Trust the first presses	One correct press is not evidence; new letters start as fairly hard and get practised. §5.2.

Decision	Alternative	Why (and where it is documented)
Softmax with temperature	Always the worst letter	Focus and variety; one parameter (T = 0.25, ×55 between difficulty 1 and 0). §7.
Gradual unlocking by frequency	All letters at once	A beginner cannot learn 26 positions at once; the order comes from data. §6.
Tab never restarts	Monkeytype-style restart	Restarting throws away the mistakes the trainer learns from. Product decision.
Wrong layout / Caps Lock are not mistakes	Count every wrong key	They have nothing to do with typing skill and would ruin the statistics. §12.
Own progress per language	One shared profile	Skill in one language says nothing about another. §6.
Go stdlib + plain JS	Frameworks	Small, readable, defensible code; nothing hidden in libraries. AGENTS.md.
Store interface	Direct database code	In memory now, PostgreSQL later by adding one file. ARCHITECTURE.md.
WebAssembly for the website	Rewrite the logic in JS	One codebase for the server and the website; nothing duplicated. ARCHITECTURE.md.
Rewards never affect measurement	—	The data stays trustworthy for evaluation. AGENTS.md.
raw wpm shown only	Store it	Familiar from Monkeytype; not part of the model.
Heatmap on the stats screen	Toggle on the typing screen	Analysis belongs next to the statistics; the typing screen stays clean.

FOR THE METHODOLOGY CHAPTER

6. How the AI-assisted workflow worked

This section describes the process itself, which can be part of the methodology: the student designed the product and made the decisions, the agent implemented and verified them.

The loop for every change

```
1. Request    the student describes what he wants, often by voice or with a screenshot
2. Clarify    a short plan for bigger work; real choices asked as questions
3. Implement  in small steps, following AGENTS.md
4. Check      gofmt, go vet, go test; the app is rebuilt and driven in a headless
              browser at desktop and phone sizes (screenshots, measured positions,
              no console errors)
5. Publish    a descriptive commit, push to main; CI runs the tests and redeploys
6. Report     a short summary in plain Russian: what changed, which files, how to check
7. Feedback   the student tries it on his PC and phone; the next iteration starts
```

Division of work

Student	AI agent
The idea, the target users and the “feel” (Monkeytype look, but teaching).	Turning requests into code that follows the project rules.
Product decisions: real words, one mode, no restart on Tab, what is in and out of scope.	Proposing the statistical model and explaining trade-offs; never changing the formulas without asking.
Visual judgement: consistency, what feels natural (hands, thumbs, sizes, notifications).	Tests, documentation kept in sync with the code, deployment.

Student	AI agent
Testing on real devices (Windows PC and his phone) and reporting what felt wrong.	Automated checks in a browser at several screen sizes before every push.
Final say on every change.	Explaining every change in simple words so it can be defended.

What made it work

- A written rules file, so every new session starts with the same constraints.
- Small steps and immediate deployment: every change could be seen on the phone within minutes.
- Honest feedback in the student's own words; several features only became good on the second or third try.
- Documentation as part of each task, so the thesis material grows with the code.
- Verification before reporting: what was checked, and what could not be checked (for example the blue tap flash on Android could not be reproduced in the test browser).

Points to address in the thesis

- Agree with the supervisor how AI use is described and whether the repository may stay public before the defence.
- The theoretical text is written by the student (anti-plagiarism); this log and the chat are raw material, not text to copy.
- The student must be able to explain the algorithm and the key architecture decisions in his own words; code details are shown through screenshots and the documentation.

FOR THE EVALUATION CHAPTER

7. Known limitations and open questions

- **Parameters were chosen by reasoning, not fitted to data.** Collecting real typing data (export files) is the way to test and tune them; ALGORITHM.md §11 suggests a simulation and a small user study (adaptive vs random words).
- **Time belongs to a transition, not a letter:** a slow “t→s” is recorded on “s”. Bigram statistics are a natural extension.
- **One stubborn letter blocks progress**, because a new letter opens only when all open ones are mastered.
- **Little data looks hard:** with few presses every letter is pulled towards the prior 0.7, so the heatmap is mostly red at first.
- **Phone and keyboard data measure different skills** and should not be mixed in an evaluation.
- **Storage is in memory** until PostgreSQL is added; export / import bridges the gap.
- Lowercase only (no Shift yet): English “I” appears as a lone “i”; single-letter words are kept only when they are real words.
- Raw wpm is inflated in the first second of an exercise (very little time has passed).

WRITING PLAN

8. Where this material fits in the thesis

Chapter	Material available	Still to do
Introduction, goal, scope	The idea (§1), scope and “not now” list (AGENTS.md).	Motivation in your own words.
Market analysis	Monkeytype as the look-and-feel reference; keybr as the teaching reference.	Try the sites yourself and compare (planned).
Theory	Reading list in docs/LITERATURE.md (touch typing, motor learning, adaptive practice, word frequencies).	Read and write the chapter yourself.
Design of the algorithm	docs/ALGORITHM.md: formulas, parameters, design decisions (§5 of this log).	Explain it in your own words; figures.
Implementation	docs/ARCHITECTURE.md, the project structure, the two builds, screenshots (docs/screenshots).	Choose what to show; describe the workflow (§6).
Evaluation	Unit tests (32), export files with your own typing data, the ideas in ALGORITHM.md §11.	Collect data, analyse it, maybe a small study.
Conclusions and future work	Limitations (§7); PostgreSQL, accounts, bigrams, forgetting over time.	Write after the evaluation.

GIT HISTORY

Appendix: every commit

Times are local (Europe/Warsaw). The first commit already contained the prototype from 3 October together with the rules files.

Commit	Date	Change
`40f4700`	04 Oct 01:21	Add agent rules
`3491b84`	12 Oct 11:43	Smooth caret, keyboard size slider, separate stats view
`1c206f6`	12 Oct 11:55	Publish to GitHub Pages via WebAssembly
`11f55fe`	12 Oct 12:21	Add Polish, Ukrainian and Russian
`bf3738b`	12 Oct 12:25	Pages: version every asset so browsers never mix cached files
`63690ce`	12 Oct 12:33	Interface in the practice language (en, pl, uk, ru)
`e1f33b5`	12 Oct 13:05	Livelier interface: rewards, clean-word streak, Tab without restart
`c3ff57d`	12 Oct 13:12	Start screen: logo types itself, random phrase, page slides in
`b464cef`	12 Oct 13:15	Splash: hide the header logo until the flying logo lands
`71777af`	12 Oct 13:22	Header: type/stats and language switches side by side
`ebe2022`	12 Oct 13:44	UI: centred layout, icon toolbar, finger hands, first-visit intro, shortcuts
`4042276`	12 Oct 13:51	Header: icon type/stats switch, switches always centred
`4b2787a`	12 Oct 14:55	Toasts: top right stack with a time-left bar
`7972310`	12 Oct 15:16	Reset button: back to the first visit
`04cf042`	12 Oct 15:31	Toast kinds with their own look; caret stays put on Tab
`794cc5b`	12 Oct 15:34	Smooth transitions instead of jumps
`54e857a`	12 Oct 16:29	Status pills under the letters; warning no longer moves the page
`15618ca`	12 Oct 17:09	Phone layout: content under the header, compact messages
`a908165`	12 Oct 18:32	Phone: keyboard hidden by default, content centred when it is hidden
`795fc23`	12 Oct 18:44	Heatmap icon as tinted keys; calmer switch animation
`5d94f11`	12 Oct 18:52	Higher-contrast switches, no tap highlight on phones
`7fa0082`	12 Oct 19:02	Export and import of statistics as a JSON file
`9e5e1da`	12 Oct 19:23	One look for switches and buttons, centred content, no loading bar
`7cbbfe3`	12 Oct 21:24	Tab icon: a yellow key with the home-row bump
`8093ed1`	12 Oct 21:30	Streak looks like the other numbers and heats up as it grows
`1b8ed6c`	12 Oct 21:39	Heatmap moves to the stats screen as a grey difficulty keyboard
`5424e00`	12 Oct 21:58	Logo with key icon and motto, raw wpm, shortcut hints, new hands
`727fc3c`	12 Oct 22:08	Start phrase lands on the logo, keyboard presets, no scrolling, new hands
`8a27ea3`	12 Oct 22:15	Natural thumbs, bigger keyboard presets, phrase over the splash logo
`cb3f9f4`	12 Oct 22:25	Messages become part of the interface instead of pop-ups
`3c27db2`	12 Oct 22:28	Flat background; the start phrase only moves, never resizes
`cdb32f7`	12 Oct 22:39	README for visitors: short, with screenshots; details move to a user guide